10(a): Create a PWA with a Web App Manifest including App Name, Icons, Start URL, and Display Mode.
10(b): Implement Push Notifications in a PWA.

Create the ASP.NET Core MVC Project

1. Create the Project

Create a new project → ASP.NET Core Web App (Model-View-Controller) → Next, **Project Name:** MyPWA

Create the Manifest

2. Create the Manifest Folder: wwwroot → Right-click → Add → New Folder. Name it: manifest

3. Create manifest.json

Right-click the manifest folder → Add → New Item → JSON File. Name it: manifest.json
Location: wwwroot/manifest/manifest.json

```
{
  "name": "My Progressive Web App",
  "short_name": "MyPWA",
  "start_url": "/",
  "display": "standalone",
  "background_color": "#ff0000",
  "theme_color": "#0000ff",
  "icons": [
    {
      "src": "/icon/myicon.png",
      "type": "image/png"
    }
  ]
}
```

Create the Icon Folder: Inside wwwroot: Right-click wwwroot → Add → New Folder. Name it: icon

Add your image: wwwroot/icon/myicon.png

Configure the Application

Program.cs

```
var builder = WebApplication.CreateBuilder(args);
builder.Services.AddControllersWithViews();
var app = builder.Build();
if (!app.Environment.IsDevelopment())
{
    app.UseExceptionHandler("/Home/Error");
    app.UseHsts();
}
app.UseHttpsRedirection();

app.UseStaticFiles();
app.UseRouting();
app.UseAuthorization();
app.MapControllerRoute(
    name: "default",
    pattern:
"{controller=Home}/{action=Index}/{id?}");
app.Run();
```

UseStaticFiles() allows the application to serve manifest.json and myicon.png from wwwroot.

Read Manifest Values in _Layout.cshtml

Open _Layout.cshtml

```
<!DOCTYPE html>
<html lang="en">

<head>

  <meta charset="utf-8" />

  <meta name="viewport" content="width=device-width,
initial-scale=1.0" />
  <title>@ViewData["Title"] - MyPWA</title>
  <link rel="manifest" href="/manifest/manifest.json" />
<!-- PWA Manifest -->
  <link rel="icon" href="/icon/myicon.png" />    <!--
Favicon -->
  <style>
      body {
        margin: 20px;
        font-family: Arial, sans-serif;
        Default color
        background-color: white;
        color: black;
      }
  </style>
</head>

<body>
  @RenderBody()
  <script>
      fetch('/manifest/manifest.json')          // Read
manifest.json
        .then(response => {
          if (!response.ok) {                   // Check
whether manifest was loaded
            throw new Error("Manifest not found");
          }
          return response.json();
        })

        .then(manifest => {
          console.log("Manifest loaded:", manifest);
          // BACKGROUND COLOR
          if (manifest.background_color) {
            document.body.style.backgroundColor =
              manifest.background_color;
          }
          // TEXT COLOR
          if (manifest.background_color === "#ff0000") {
            document.body.style.color = "white";
```

```
        }
        else {
            document.body.style.color = "black";
        }
        // THEME COLOR
        if (manifest.theme_color) {
            let theme =
                document.querySelector(
                    'meta[name="theme-color"]'
                );
            if (!theme) {
                theme =
                    document.createElement("meta");
                theme.name = "theme-color";
                document.head.appendChild(theme);
            }
            theme.content =
                manifest.theme_color;
        }
        //Result of above <meta name="theme-color"
content="#0000ff">

        // PAGE TITLE
        if (manifest.name) {
            document.title =
                manifest.name;
        }

        // DESCRIPTION
        if (manifest.description) {
            let description =
                document.createElement("meta");
            description.name =
                "description";
            description.content =
                manifest.description;
            document.head.appendChild(
                description
            );
        }
        // LANGUAGE
        if (manifest.lang) {
            document.documentElement.lang =
                manifest.lang;
        }


        // ICON
        if (manifest.icons &&
            manifest.icons.length > 0) {
            let icon =
                document.querySelector(
                    'link[rel="icon"]'
                );
            if (!icon) {
                icon =
                    document.createElement("link");
                icon.rel = "icon";
                document.head.appendChild(icon);
            }
            icon.href =
                manifest.icons[0].src;
        }
        // OTHER MANIFEST PROPERTIES
        console.log("Short Name:", manifest.short_name);
        console.log("Start URL:", manifest.start_url);
        console.log("Scope:", manifest.scope);
        console.log("Display:", manifest.display);
        console.log("Orientation:", manifest.orientation);
    })
    .catch(error => {
        console.error(
            "Error loading manifest:",
            error
        );
    });
</script>

@await RenderSectionAsync(
    "Scripts",
    required: false
)
</body>
</html>
```

Part F — Create a Page to Display the Values

8. Open Index.cshtml

```
@{
    ViewData["Title"] = "PWA Demo";
}

<h1>Progressive Web App Demo</h1>
<p>
    The following settings are loaded from
manifest.json:
</p>

<ul>
    <li>App Name</li>
    <li>Background Color</li>
    <li>Theme Color</li>
    <li>Icon</li>
</ul>
```

Part G — Run the Application

Implement Push Notifications in a PWA

For an existing ASP.NET Core MVC PWA, the simplest demonstration of Push Notifications requires three parts:

- Service Worker → receives notifications.
- JavaScript → asks permission and subscribes the user.
- Server → sends the notification.

1. Add a Service Worker

Create:

wwwroot
└── service-worker.js

```
self.addEventListener("push", function (event) {

    const data = event.data
        ? event.data.text()
        : "New Notification";

    event.waitUntil(

        self.registration.showNotification(
            "MyPWA",
            {
                body: data,
                icon: "/icon/myicon.png"
            }
        )

    );

});
```

2. Register it in _Layout.cshtml

Add this inside your existing <script>:

```
// Register Service Worker
if ("serviceWorker" in navigator) {
    navigator.serviceWorker.register(
        "/service-worker.js"
    )
    .then(function () {
        console.log("Service Worker registered");
    })
    .catch(function (error) {
        console.log(
            "Service Worker error:",
            error
        );
    });
}
```

3. Ask for notification permission

```
// Ask for notification permission

Notification.requestPermission()
    .then(function (permission) {

        console.log(
            "Notification permission:",
            permission
        );
    });
```

The browser will show: “Allow notifications?” Select Allow.

4. Test a notification

For a simple test, add a button to Index.cshtml:

```
<button onclick="showNotification()">
  Send Notification
</button>
<script>
    function showNotification() {
    Notification.requestPermission()
      .then(function(permission) {
        if (permission === "granted") {
          new Notification(
            "MyPWA",
            {
              body: "Hello! This is a PWA notification.",
              icon: "/icon/myicon.png"
            }
          );
        }
      });
  }
</script>
```

Click Send Notification. You should receive: